

WHY DO YOU LIKE FUNCTIONAL
PROGRAMMING SO MUCH? WHAT
DOES IT ACTUALLY GET YOU?

TAIL RECURSION IS
ITS OWN REWARD.



<http://xkcd.com/1270/>

CS 252:

Advanced Programming Language Principles



Lambdas & Higher-Order Functions

Prof. Tom Austin

San José State University

Lambdas

- Based on the lambda calculus
- Analogous to anonymous classes in Java



Lambda Example

```
Prelude> (\x -> x+1) 1
```

2

```
Prelude> (\x y -> x*y) 2 3
```

6

```
Prelude>
```

Function composition

$$f(g(x))$$

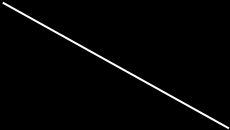
can be rewritten as

$$(f \cdot g)(x)$$

Points-free style

`inc x = x + 1`

`incByTwo = inc . inc`



Points-free: no
function argument

Lambdas & Function Composition

```
Prelude> let f = (\x -> x - 5)  
           . (\y -> y * 2)
```

```
Prelude> f 7  
9
```

```
Prelude> let f = (\x y -> x - y)  
           . (\z -> z * (-1))
```

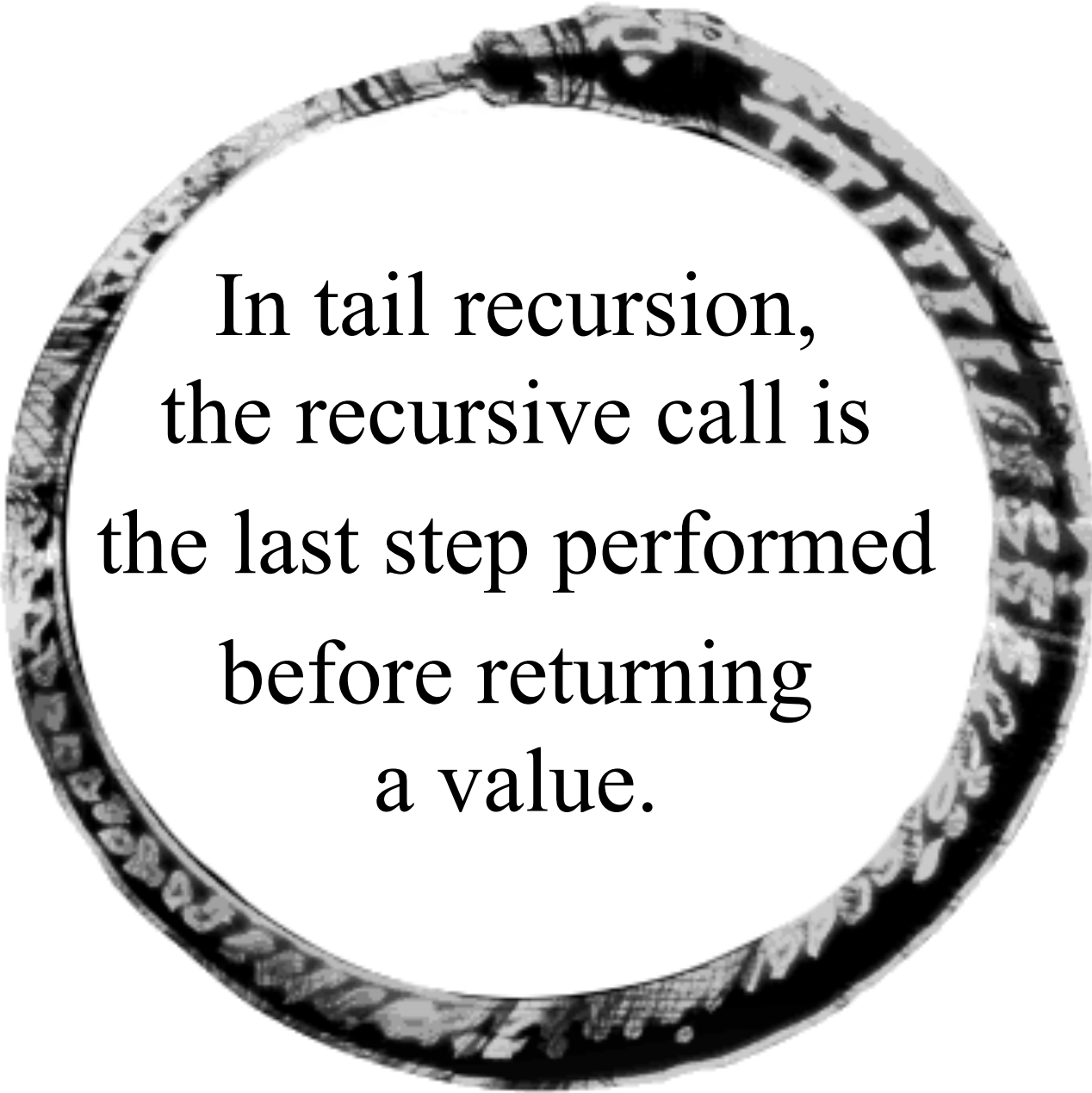
```
Prelude> f 3 4  
-7
```

Tail Recursion



Iterative solutions tend to be more efficient than recursive solutions.

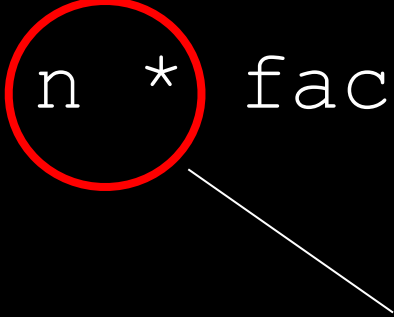
However, compilers are very good at optimizing a tail recursive functions.

A circular, textured frame, possibly made of wood or metal with a carved pattern, surrounds the text. The frame is slightly irregular and has a dark, possibly black or dark brown, finish with lighter, carved details.

In tail recursion,
the recursive call is
the last step performed
before returning
a value.

Is this function tail-recursive?

```
public int factorial(int n) {  
    if (n==1) return 1;  
    else {  
        return n * factorial(n-1);  
    }  
}
```



No: the last step is
multiplication

Is this function tail-recursive?

```
public int factorialAcc(int n, int acc)
{
    if (n==1) return acc;
    else {
        return factorialAcc(n-1, n*acc);
    }
}
```

Yes: the recursive
step is the last thing
we do

Which version is tail-recursive?

```
fact :: Integer -> Integer
```

```
fact 1 = 1
```

```
fact n = n * (fact $ n - 1)
```

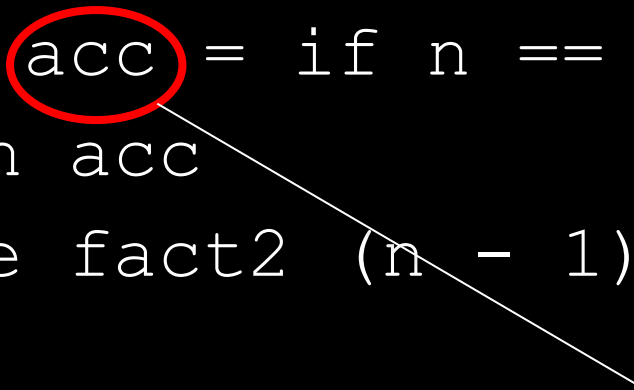
```
fact' :: Integer -> Integer -> Integer
```

```
fact' 0 acc = acc
```

```
fact' n acc = fact' (n - 1) (n * acc)
```

Is this version tail-recursive?

```
fact2 :: Integer -> Integer -> Integer
fact2 n acc = if n == 0
  then acc
  else fact2 (n - 1) (n * acc)
```



This argument is called an “accumulator” – common design pattern to make your functions tail-recursive.

Higher-order functions



Programs as Functions

Functional languages treat programs as mathematical functions.

***Definition:** A function is a rule that associates to each x from some set X of values a unique y from a set of Y values.*

Definition: A function is a rule that associates to each x from some set X of values a unique y from a set of Y values.

f is the name of
the function

$$y = f(x)$$

Definition: A function is a rule that associates to each x from some set X of values a unique y from a set of Y values.

x is a variable in
the set X

$$y = f(x)$$

X is the *domain* of f .
 $x \in X$ is the *independent*
variable.

Definition: A function is a rule that associates to each x from some set X of values a unique y from a set of Y values.

$$y = f(x)$$

y is a variable in
the set Y

Y is the *range* of f .
 $y \in Y$ is the *dependent*
variable.

Qualities of Functional Programming

1. Functions clearly distinguish
 - incoming values (parameters)
 - outgoing values (results)
2. No assignment
3. No loops
4. Return value depends only on params
5. *Functions are first class values*

Functions are first-class data values,
so we can:

- Pass as arguments to a function
- Return from a function
- Construct new functions dynamically

A function that either takes a
function as a parameter or returns a
function as its result is a
higher-order function

Consider:

$$\text{addNums } x \ y = x + y$$

I mean to type

$$3 \ * \ \text{addNums} \ 5 \ 2$$

But accidentally type

$$3 \ * \ \text{addNums} \ 52$$

What happens?

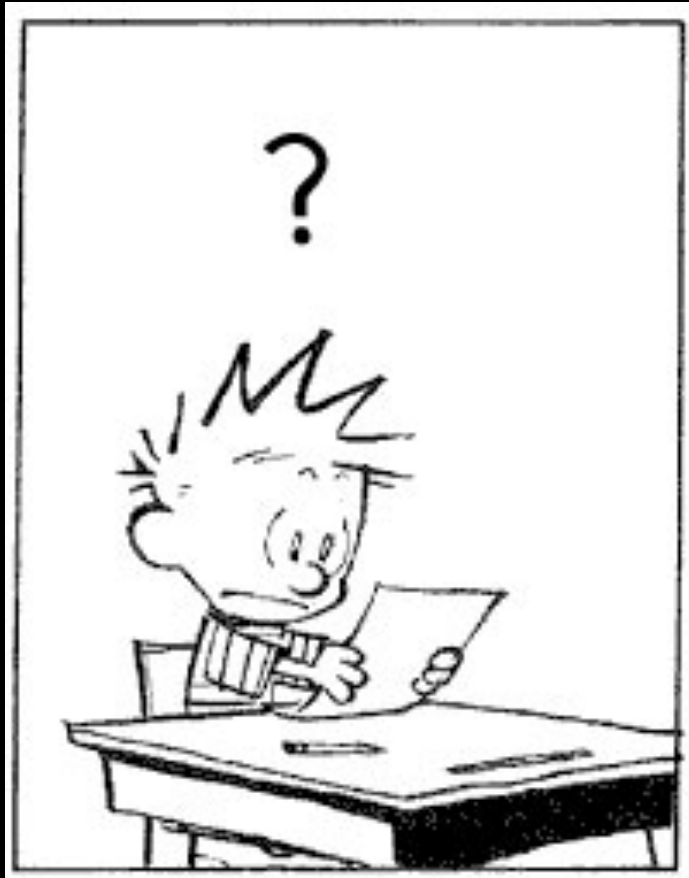
Non type-variable argument in the
constraint: Num (a -> a)

(Use FlexibleContexts to permit
this)

When checking that 'it' has the
inferred type

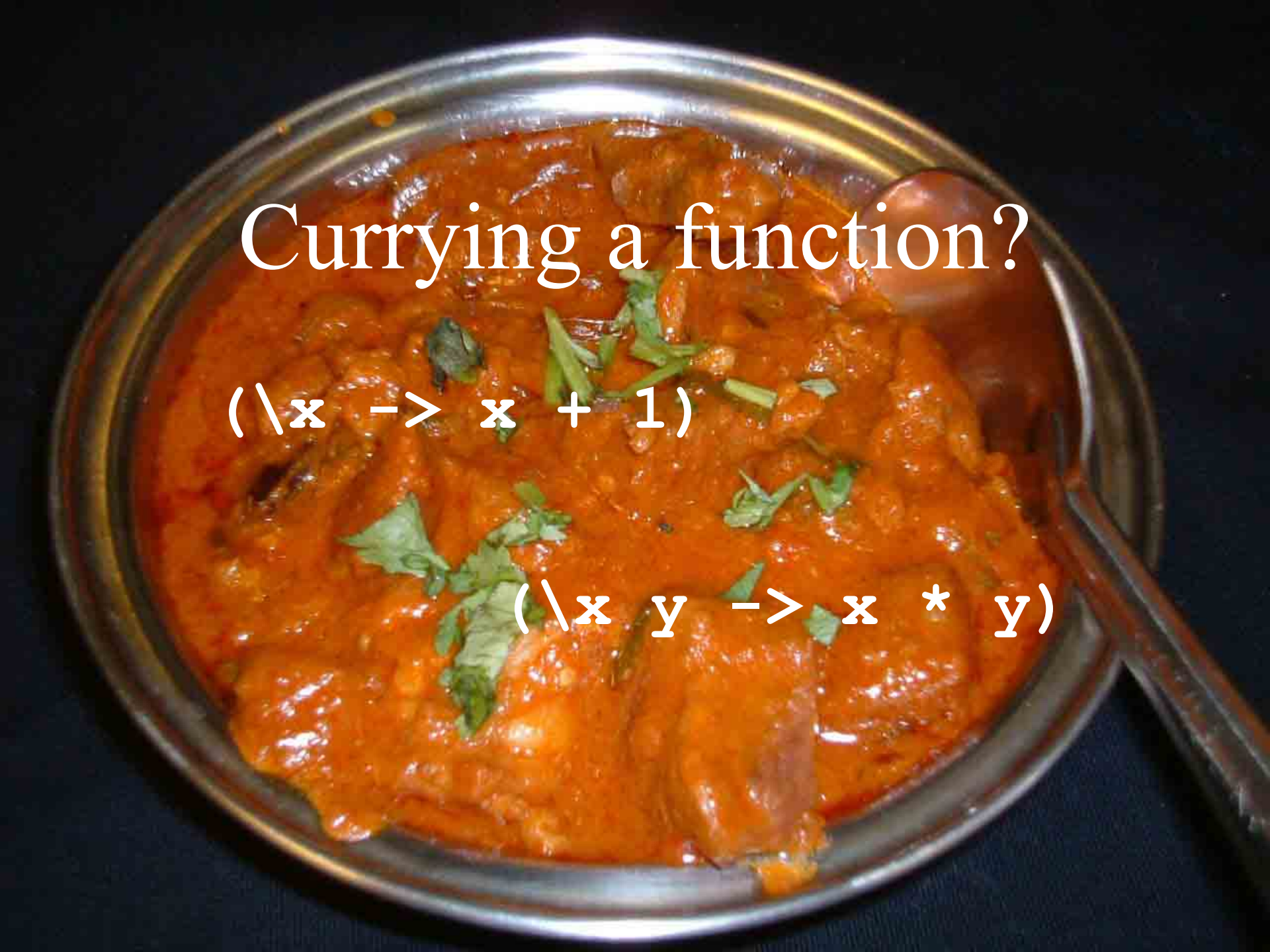
it :: forall a.

(Num a, Num (a -> a)) => a -> a



Why does
Haskell
give such
strange error
messages?

The answer is that Haskell
curries functions.

A top-down view of a round, metallic bowl filled with a thick, orange-red curry. The curry has a chunky texture with visible pieces of meat or vegetables. It is garnished with several sprigs of fresh green cilantro leaves. A metal spoon is partially submerged in the curry on the right side of the bowl. The background is dark and out of focus.

Currying a function?

$(\backslash x \rightarrow x + 1)$

$(\backslash x \ y \rightarrow x * y)$

Function currying

Transform a function
w/ multiple arguments
into multiple functions



Haskell Brooks Curry

Function currying

- Note the type of our Haskell function
 - `addNums :: Num a => a -> a -> a`
- `addNums` is a function that takes in a number *and returns a function that takes another number*

Motivation for higher order functions (in-class)

Higher order functions

```
map :: (a -> b) -> [a] -> [b]
```

```
zipWith :: (a -> b -> c) ->
           [a] -> [b] -> [c]
```

```
filter :: (a -> Bool) -> [a] -> [a]
```

map

```
Prelude> map (+1) [1,2,3]
[2,3,4]
Prelude> map show [1,2,3]
["1","2","3"]
Prelude> let squares = map (\x -> x*x) [1..]
Prelude> take 10 squares
[1,4,9,16,25,36,49,64,81,100]
Prelude>
```

zipWith

```
Prelude> map (+) [1,2,3] [3,2,1]
```

```
<interactive>:8:1: error:
```

- Couldn't match expected type '[a1] -> t'
with actual type '[a0 -> a0]'
- The function 'map' is applied to three arguments,
but its type '(a0 -> a0 -> a0) -> [a0] -> [a0 -> a0]'
has only two
In the expression: map (+) [1, 2, 3] [3, 2, 1]
In an equation for 'it': it = map (+) [1, 2, 3]
[3, 2, 1]
- Relevant bindings include it :: t
(bound at <interactive>:8:1)

```
Prelude> zipWith (+) [1,2,3] [3,2,1]
```

```
[4,4,4]
```

```
Prelude>
```

filter

```
Prelude> filter (>0) [-5..5]
```

```
[1,2,3,4,5]
```

```
Prelude> filter even $
```

```
    map read ["1","2","3","4","5"]
```

```
[2,4]
```

```
Prelude>
```

Lab 4: Higher order functions

Available in Canvas and on the course website.